

Analyze Visium H&E data

Visium spots based purely on the tissue image

This tutorial shows how to apply Squidpy for the analysis of Visium spatial transcriptomics data.

https://squidpy.readthedocs.io/en/stable/notebooks/tutorials/tutorial_visium_hne.html

Data

dataset portal <https://support.10xgenomics.com/spatial-gene-expression/datasets>

```
In [1]: import numpy as np
import pandas as pd
import os, sys

import anndata as ad
import scanpy as sc
import squidpy as sq
import liana as li
from pathlib import Path

ROOT0 = Path("/home/flavio/uv/gwas/")
ROOT_SRC = ROOT0 / "src"

sys.path.append(str(ROOT_SRC))

from libs.Basic import create_dir, pdwritecsv, pdreadcsv
from libs.matplotlib_figure_style import *

import matplotlib.pyplot as plt

sc.logging.print_header()
print(f"squidpy=={sq.__version__}")

squidpy==0.1.dev3+gafae60bd3.d20261007
```

```
In [2]: # load the pre-processed dataset
img = sq.datasets.visium_hne_image()
```

```
In [3]: root0 = Path('/home/flavio/uv/squidpy/')
root_data = root0 / 'data'
root_figure = root0 / 'figures'
root_result = root0 / 'results'

os.listdir(root_figure)
```

```
Out[3]: ['liana_cluster_dotplot.png',
'spatial_clusters.png',
'features_vs_gene_clusters.png',
'liana_spatial_LR.png']
```

```
In [ ]: filename_vis = root_data / "visium_hne.h5ad"
force=False
```

```

if filename_vis.exists() and not force:
    adata = ad.read_h5ad(filename_vis)
else:
    adata = sq.datasets.visium_hne_adata()

```

Gene count per cell

```

In [5]: df_de = adata.var
print(df_de.shape)

int(df_de.n_cells_by_counts.max())

```

(18078, 16)

Out[5]: 2698

```

In [6]: df_de.head(5).T

```

Out[6]:

	Xkr4	Sox17	M
gene_ids	ENSMUSG00000051951	ENSMUSG00000025902	ENSMUSG00000003
feature_types	Gene Expression	Gene Expression	Gene Expression
genome	mm10	mm10	r
mt	False	False	
n_cells_by_counts	233	298	
mean_counts	0.093032	0.128243	1.37
log1p_mean_counts	0.088955	0.120662	0.86
pct_dropout_by_counts	91.363973	88.954781	34.21
total_counts	251.0	346.0	3
log1p_total_counts	5.529429	5.849325	8.21
n_cells	233	298	
highly_variable	False	False	
highly_variable_rank	NaN	NaN	
means	0.093378	0.12872	1.37
variances	0.098832	0.156108	2.16
variances_norm	0.815019	0.931378	0.85

```

In [7]: df_de[df_de.mt==True].head(5).T

```

Out[7]:

	mt-Nd1	mt-Nd2	m
gene_ids	ENSMUSG00000064341	ENSMUSG00000064345	ENSMUSG0000000064341
feature_types	Gene Expression	Gene Expression	Gene Expression
genome	mm10	mm10	mm10
mt	True	True	True
n_cells_by_counts	2698	2698	2698
mean_counts	338.398438	239.262039	706.500000
log1p_mean_counts	5.827175	5.48173	6.550000
pct_dropout_by_counts	0.0	0.0	0.0
total_counts	912999.0	645529.0	1907000.0
log1p_total_counts	13.724491	13.377827	14.460000
n_cells	2688	2688	2688
highly_variable	False	False	False
highly_variable_rank	NaN	NaN	NaN
means	339.500744	240.046875	709.230000
variances	19001.217714	7304.72724	67488.240000
variances_norm	0.586821	0.412852	0.586821

```
In [8]: for i, key in enumerate(adata.obs.keys()):
        print(i, key)
```

```
0 in_tissue
1 array_row
2 array_col
3 n_genes_by_counts
4 log1p_n_genes_by_counts
5 total_counts
6 log1p_total_counts
7 pct_counts_in_top_50_genes
8 pct_counts_in_top_100_genes
9 pct_counts_in_top_200_genes
10 pct_counts_in_top_500_genes
11 total_counts_mt
12 log1p_total_counts_mt
13 pct_counts_mt
14 n_counts
15 leiden
16 cluster
```

```
In [9]: for i, key in enumerate(adata.obsm.keys()):
        print(i, key)
```

```
0 X_pca
1 X_umap
2 spatial
```

In []:

let's visualize cluster annotation in spatial context with `squidpy.pl.spatial_scatter()`.

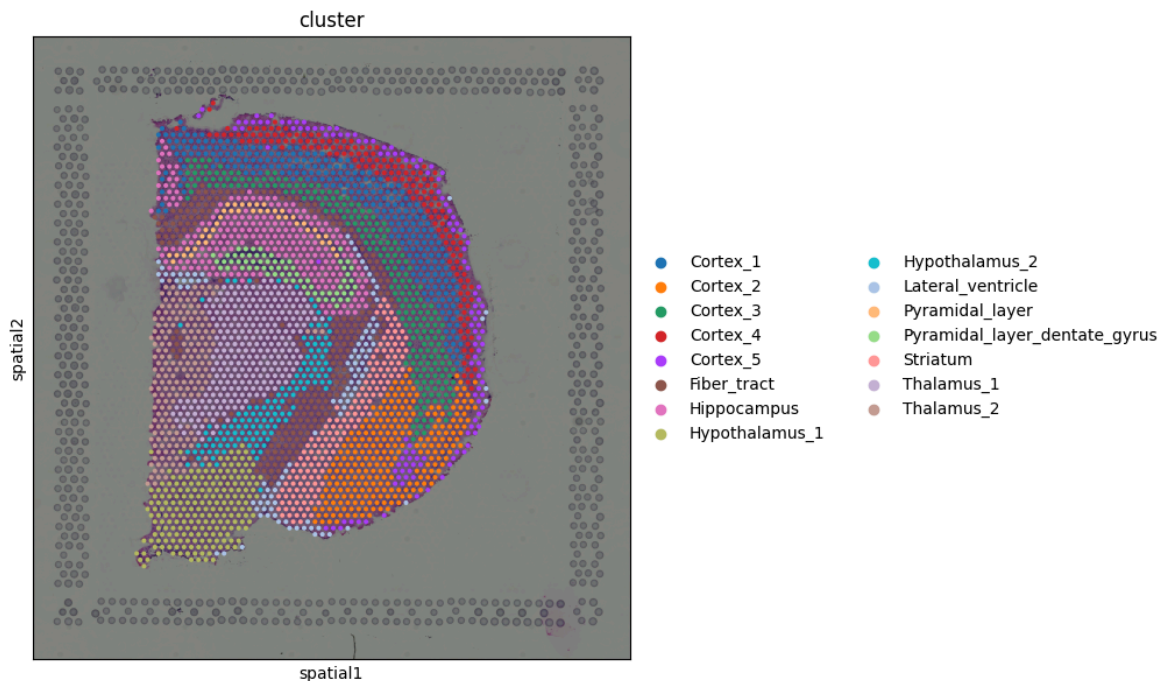
```
In [ ]: plt.rcParams['figure.figsize'] = (10, 7)
# plt.rcParams['figure.dpi'] = 240

ax = sq.pl.spatial_scatter(adata, color="cluster", return_ax=True)

filename = root_figure / "spatial_clusters.png"

if not filename.exists():
    ax.figure.savefig(filename, dpi=300, bbox_inches="tight")
    print(f"Plot saved: {filename}")
```

Plot saved: /home/flavio/uv/squidpy/figures/spatial_clusters.png



In []:

Image feature

Visium datasets contain high-resolution images of the tissue that was used for the gene extraction. Using the function `squidpy.im.calculate_image_features()` you can calculate image features for each Visium spot and create a `obs x features` matrix in `adata` that can then be analyzed together with the `obs x gene` gene expression matrix.

By extracting image features we are aiming to get both similar and complementary information to the gene expression values. Similar information is for example present in the case of a tissue with two different cell types whose morphology is different. Such cell type information is then contained in both the gene expression values and the tissue image features.

Squidpy contains several feature extractors and a flexible pipeline of calculating features of different scales and sizes. There are several detailed examples of how to use

`squidpy.im.calculate_image_features()`. Extract image features provides a good starting point for learning more.

In []:

extract summary features at different crop sizes and scales

to allow the calculation of multi-scale features and segmentation features. For more information on the summary features, also refer to [Extract summary features](#).

```
In [11]: for i, key in enumerate(adata.obsm.keys()):
          print(i, key)
```

```
0 X_pca
1 X_umap
2 spatial
```

```
In [ ]: force=False

if "features" not in adata.obsm.keys() or force:

    # calculate features for different scales (higher value means more co
    for scale in [1.0, 2.0]:
        feature_name = f"features_summary_scale{scale}"
        print(">>>", feature_name)
        sq.im.calculate_image_features(
            adata,
            img.compute(),
            features="summary",
            key_added=feature_name,
            n_jobs=12,
            scale=scale,
        )

    # combine features in one dataframe
    adata.obsm["features"] = pd.concat(
        [adata.obsm[f] for f in adata.obsm.keys() if "features_summary" i
    )
    # make sure that we have no duplicated feature names in the combined
    adata.obsm["features"].columns = ad.utils.make_index_unique(
        adata.obsm["features"].columns
    )
```

```
>>> features_summary_scale1.0
0%|          | 0/2688 [00:00<?, ?/s]
>>> features_summary_scale2.0
0%|          | 0/2688 [00:00<?, ?/s]
```

extracted image features to compute a new cluster annotation

We can use the extracted image features to compute a new cluster annotation. This could be useful to gain insights in similarities across spots based on image morphology.

"features_cluster" are the "leiden" regions, what do they represent?

In that tutorial, `features_cluster` is a clustering of the **Visium spots based purely on the tissue image**, not on gene expression.

What's going on:

- `adata.obsm["features"]` comes from `sq.im.calculate_image_features(...)`. For each spot, Squidpy crops the H&E (or fluorescence) image around the spot's coordinates and computes numeric descriptors of that crop. The columns are prefixed by feature type: `summary_*`, `histogram_*`, `texture_*`, `segmentation_*`.
- `like="summary"` keeps only the summary-statistics features — per channel, things like mean, standard deviation, and quantiles of pixel intensity in the crop. So each spot is represented by a small vector describing how dark/light, how variable, and how colored its local patch of tissue looks.
- `cluster_features` then does the standard scanpy pipeline (scale → PCA → kNN graph → Leiden) on that matrix, treating spots as observations and image statistics as "genes".

So each Leiden label ("0", "1", "2", ...) is an arbitrary integer identifying a group of spots whose local histology looks alike in terms of staining intensity and contrast — e.g. densely packed, darkly stained nuclei-rich regions vs. pale, sparse white matter vs. background/edge spots. The labels carry no biological name and no spatial constraint; spots in the same cluster can be scattered across the section.

The point of the tutorial is the comparison with `adata.obs["cluster"]` (the expression-derived, anatomically annotated clusters): plotting both side by side shows how much of the anatomy is recoverable from image appearance alone. You typically see partial correspondence — large-scale structures like cortex vs. white matter vs. hippocampus separate reasonably well, while finer expression-defined subregions collapse or split differently. Adding `texture` or `segmentation` features (nuclei counts, cell density) usually improves that agreement.

```
In [13]: # helper function returning a clustering
def cluster_features(features: pd.DataFrame, like=None) -> pd.Series:
    """
    Calculate leiden clustering of features.
    Specify filter of features using `like`.
    """
    # filter features
    if like:
        features = features.filter(like=like)
    # create temporary adata to calculate the clustering
    adata = ad.AnnData(features)
    # important - feature values are not scaled, so need to scale them be
    sc.pp.scale(adata)
    # calculate leiden clustering
    sc.pp.pca(adata, n_comps=min(10, features.shape[1] - 1))
    sc.pp.neighbors(adata)
    sc.tl.leiden(adata)
```

```
return adata.obs["leiden"]
```

calculate feature clusters

```
In [ ]: force=False

if "features_cluster" not in adata.obs.keys() or force:
    adata.obs["features_cluster"] = cluster_features(adata.obsm["features
```

/tmp/ipykernel_59001/1792408318.py:17: FutureWarning: The `igraph` implementation of leiden clustering is *orders of magnitude faster*. Set the flavor argument to (and install if needed) 'igraph' to use it.
In the future, the default backend for leiden will be igraph instead of leidenalg. To achieve the future defaults please pass: `flavor='igraph'` and `n\_iterations=2`. `directed` must also be `False` to work with igraph's implementation.

```
In [15]: for i, key in enumerate(adata.obs.keys()):
        print(i, key)
```

```
0 in_tissue
1 array_row
2 array_col
3 n_genes_by_counts
4 loglp_n_genes_by_counts
5 total_counts
6 loglp_total_counts
7 pct_counts_in_top_50_genes
8 pct_counts_in_top_100_genes
9 pct_counts_in_top_200_genes
10 pct_counts_in_top_500_genes
11 total_counts_mt
12 loglp_total_counts_mt
13 pct_counts_mt
14 n_counts
15 leiden
16 cluster
17 features_cluster
```

```
In [16]: dfc = adata.obs['cluster']
        print(type(dfc), dfc.shape)
        dfc.head(6)
```

```
<class 'pandas.core.series.Series'> (2688,)
```

```
Out[16]: AAACAAGTATCTCCCA-1      Cortex_2
         AAACAATCTACTAGCA-1      Cortex_5
         AAACACCAATAACTGC-1      Thalamus_2
         AAACAGAGCGACTCCT-1      Cortex_5
         AAACCGGGTAGGTACC-1      Thalamus_2
         AAACCGTTCGTCCAGG-1      Hypothalamus_2
         Name: cluster, dtype: category
         Categories (15, object): ['Cortex_1', 'Cortex_2', 'Cortex_3', 'Cortex_4', ..., 'Pyramidal_layer_dentate_gyrus', 'Striatum', 'Thalamus_1', 'Thalamus_2']
```

There are 257 clusters for Cortex 2

```
In [17]: df2 = dfc[dfc=='Cortex_2']
print(df2.shape)
df2.head(3)
```

```
(257,)
```

```
Out[17]: AAACAAGTATCTCCCA-1    Cortex_2
AAATAGGGTGCTATTG-1    Cortex_2
AACACACGCTCGCCGC-1    Cortex_2
Name: cluster, dtype: category
Categories (15, object): ['Cortex_1', 'Cortex_2', 'Cortex_3', 'Cortex_4', ..., 'Pyramidal_layer_dentate_gyrus', 'Striatum', 'Thalamus_1', 'Thalamus_2']
```

```
In [18]: smat = adata.X
print(smat.shape)
type(smat)
```

```
(2688, 18078)
```

```
Out[18]: scipy.sparse._csr.csr_matrix
```

```
In [19]: # obs: 'in_tissue', 'array_row', 'array_col', 'n_genes_by_counts', 'loglp'
# var: 'gene_ids', 'feature_types', 'genome', 'mt', 'n_cells_by_counts',

nrows=4
ncols=2
fig, axes = plt.subplots(nrows, ncols, figsize=(12, 8), sharex=True, sharey=True)

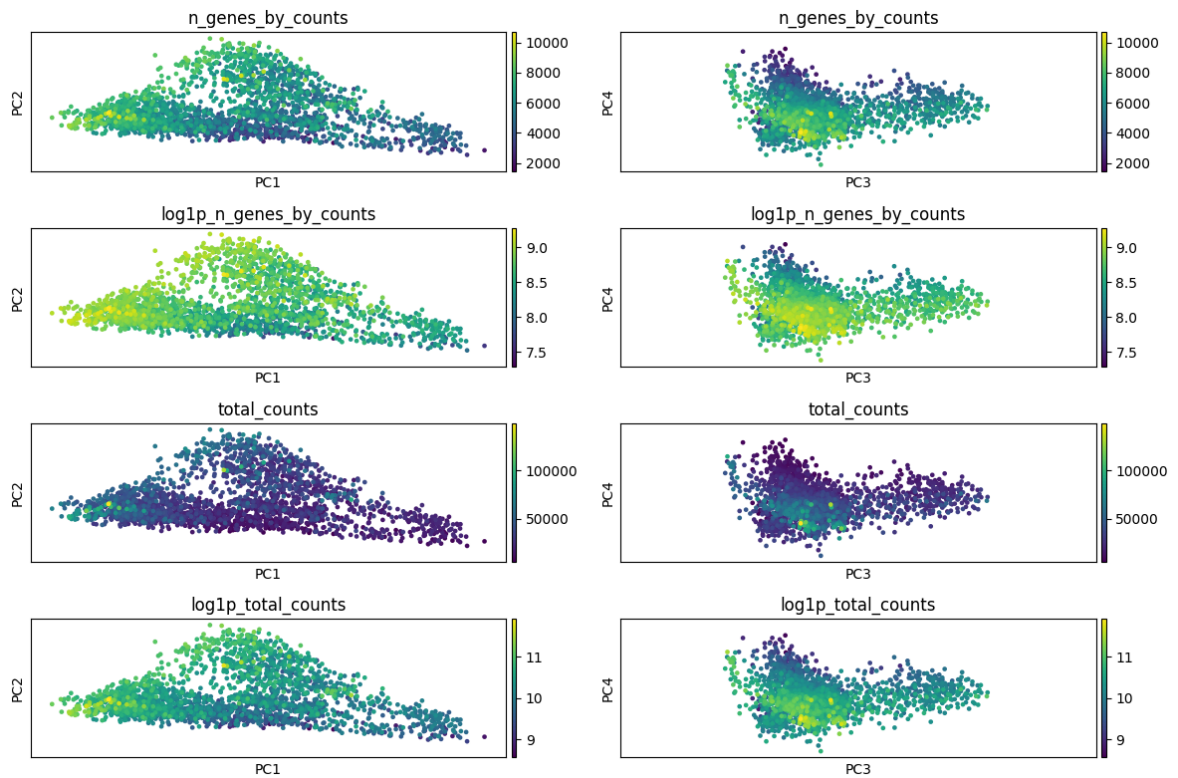
nrow=0; ncol=1
measures = ["n_genes_by_counts", "loglp_n_genes_by_counts", "total_counts"]
for nrow in range(1, nrows+1):
    for ncol in range(1, ncols+1):
        ax = axes[nrow-1, ncol-1]

        measure = measures[nrow-1]
        dimensions = (0, 1) if ncol==1 else (2,3)

        print(nrow, ncol, measure, dimensions)
        sc.pl.pca(adata, color=measure, dimensions=dimensions, ax=ax, show_legend=True)

plt.tight_layout()
```

```
1 1 n_genes_by_counts (0, 1)
1 2 n_genes_by_counts (2, 3)
2 1 loglp_n_genes_by_counts (0, 1)
2 2 loglp_n_genes_by_counts (2, 3)
3 1 total_counts (0, 1)
3 2 total_counts (2, 3)
4 1 loglp_total_counts (0, 1)
4 2 loglp_total_counts (2, 3)
```

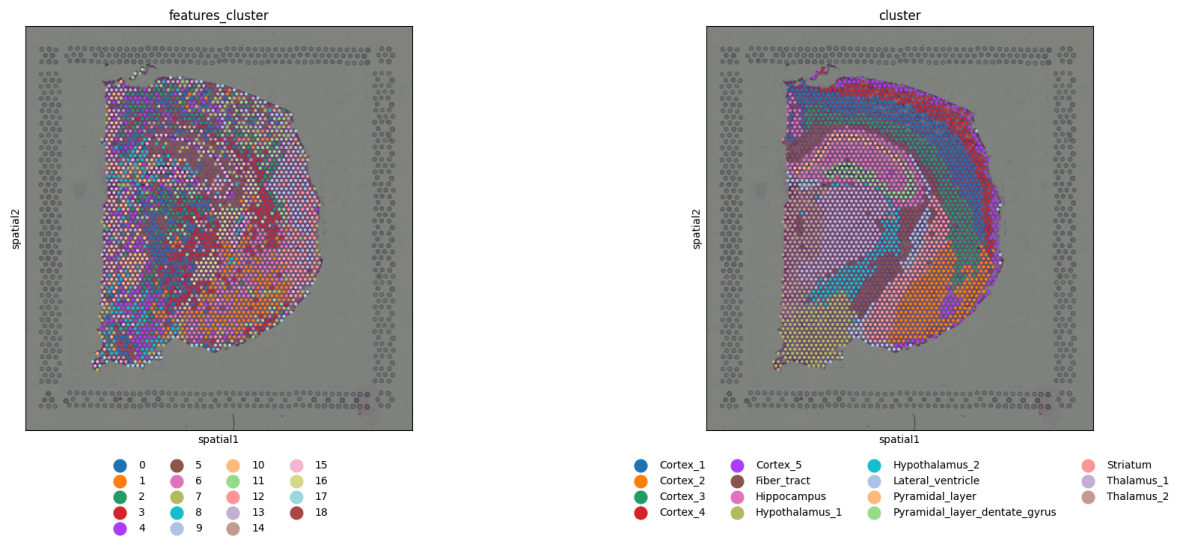
In []:

compare feature and gene clusters

```
sq.pl.spatial_scatter(adata, color=["features_cluster", "cluster"])
```

```
In [20]: ax = sq.pl.spatial_scatter(
    adata,
    color=["features_cluster", "cluster"],
    return_ax=True,
)
axs = [axs] if not isinstance(axs, (list, tuple)) else axs

for ax in axs:
    leg = ax.get_legend()
    if leg is None:
        continue
    handles = leg.legend_handles
    labels = [t.get_text() for t in leg.get_texts()]
    leg.remove()
    ax.legend(
        handles, labels,
        loc="upper center", bbox_to_anchor=(0.5, -0.05),
        ncol=4, frameon=False, markerscale=2,
    )
```



Comparing gene and feature clusters, we notice that in some regions, they look very similar, like the cluster Fiber_tract, or clusters around the Hippocampus seems to be roughly recapitulated by the clusters in image feature space. In others, the feature clusters look different, like in the cortex, where the gene clusters show the layered structure of the cortex, and the features clusters rather seem to show different regions of the cortex.

This is only a simple, comparative analysis of the image features, note that you could also use the image features to e.g. compute a common image and gene clustering by computing a shared neighbors graph (for instance on concatenated PCAs on both feature spaces).

In []:

```
In [21]: arr_spatial = adata.obsm['spatial']
arr_spatial[:3, :3]
```

```
Out[21]: array([[8230, 7237],
               [4170, 1611],
               [2519, 8315]])
```

```
In [22]: dff = adata.obsm['features']
print(dff.shape)
dff.iloc[:3, :6].T
```

```
(2688, 30)
```

Out [22]:

	AAACAAGTATCTCCCA- 1	AAACAATCTACTAGCA- 1	AAACACCAATAACTGC- 1
summary_ch- 0_quantile- 0.9	132.000000	140.000000	132.000000
summary_ch- 0_quantile- 0.5	111.000000	111.000000	116.000000
summary_ch- 0_quantile- 0.1	77.000000	80.000000	89.000000
summary_ch- 0_mean	107.571140	109.815175	112.421285
summary_ch- 0_std	21.767668	25.047208	19.088559
summary_ch- 1_quantile- 0.9	102.000000	87.000000	109.000000

In []:

Liana (Saez lab - Heidelberg)

<https://github.com/saezlab/liana>

A. Cluster-level (non-spatial) LR inference

```
In [23]: li.mt.rank_aggregate(
    adata,
    groupby="cluster",           # anatomical spot clusters (15 regio
    resource_name="mouseconsensus", # mouse gene symbols
    expr_prop=0.1,
    use_raw=False,               # .raw holds counts here!
    n_perms=1000, n_jobs=8,
    verbose=True,
)
```

```
Using resource `mouseconsensus`.
Using `.X`!
/home/flavio/uv/squidpy/.venv/lib/python3.12/site-packages/liana/method/_pipe_utils/_pre.py:167: FutureWarning: The default of observed=False is deprecated and will be changed to True in a future version of pandas. Pass observed=False to retain current behavior or observed=True to adopt the future default and silence this warning.
0.18 of entities in the resource are missing from the data.
Generating ligand-receptor stats for 2688 samples and 1257 features
```

```

/home/flavio/.local/share/uv/python/cpython-3.12.12-linux-x86_64-gnu/lib/p
ython3.12/functools.py:912: UserWarning: zero-centering a sparse array/mat
rix densifies it.
/home/flavio/uv/squidpy/.venv/lib/python3.12/site-packages/liana/method/s
c/_liana_pipe.py:288: ImplicitModificationWarning: Setting element `.layer
s['scaled']` of view, initializing view as actual.
/home/flavio/uv/squidpy/.venv/lib/python3.12/site-packages/liana/method/s
c/_liana_pipe.py:293: FutureWarning: The method uns_keys is deprecated and
will be removed in the future. Use uns instead of uns_keys. (e.g. `k in ad
ata.uns` or `sorted(adata.uns)` )
/home/flavio/uv/squidpy/.venv/lib/python3.12/site-packages/liana/method/s
c/_liana_pipe.py:296: FutureWarning: The method uns_keys is deprecated and
will be removed in the future. Use uns instead of uns_keys. (e.g. `k in ad
ata.uns` or `sorted(adata.uns)` )

```

Assuming that counts were `natural` log-normalized!

Running CellPhoneDB

100%|██████████| 1000/1000 [00:05<00:00, 176.50it/s]

Running Connectome

Running log2FC

Running NATMI

Running SingleCellSignalR

source, target, ligand_complex, receptor_complex,
...magnitude_rank, specificity_rank (lower = stronger)

```

In [ ]: res = adata.uns["liana_res"]

fname = "liana_cluster_results.tsv"
filename = root_result / fname

if not filename.exists():
    pdwritecsv(res, fname, root_result, verbose=True)

```

Table saved ((190666, 13)) at '/home/flavio/uv/squidpy/results/liana_clust
er_results.tsv'

Out[]: True

```

In [39]: top_specific = res[(res.specificity_rank <= 0.01)].sort_values("magnitude

fname = "liana_cluster_top_specific.tsv"
filename = root_result / fname

if not filename.exists():
    pdwritecsv(top_specific, "liana_cluster_top_specific.tsv", root_resul

print(top_specific.shape)
top_specific.head(6).T

```

(25341, 13)

Out[39]:

	130723	65802	4024	29636	170893	
source	Pyramidal_layer	Fiber_tract	Cortex_1	Cortex_3	Thalamus_1	Co
target	Fiber_tract	Fiber_tract	Fiber_tract	Fiber_tract	Fiber_tract	Fiber
ligand_complex	App	App	App	App	App	
receptor_complex	Aplp1	Aplp1	Aplp1	Aplp1	Aplp1	
lr_means	3.613924	3.563859	3.563744	3.553724	3.532249	3.5
cellphone_pvals	0.0	0.0	0.0	0.0	0.0	
expr_prod	12.993496	12.605726	12.604836	12.527227	12.360901	12.3
scaled_weight	1.206099	1.088152	1.087882	1.064276	1.013686	1.0
lr_logfc	0.799214	0.741875	0.726971	0.704703	0.690519	0.6
spec_weight	0.006111	0.005929	0.005928	0.005892	0.005814	0.
lrscore	0.906885	0.905598	0.905595	0.90533	0.904756	0.9
specificity_rank	0.001945	0.003135	0.003553	0.004233	0.004702	0.
magnitude_rank	0.0	0.0	0.0	0.0	0.0	



In []:

Save complete h5ad

In []:

```
force=False

if not filename_vis.exists() or force:
    adata.write_h5ad(filename_vis)
    print(f"Data written to {filename_vis}")
```

Data written to /home/flavio/uv/squidpy/data/visium_hne.h5ad

In []:

B. Spatially-resolved LR (per spot, no clusters needed)

- Spot centre spacing here is ~137 px (~100 um); bandwidth=200 px ~ 1-2 rings of neighbours.

In [27]:

```
li.ut.spatial_neighbors(adata, bandwidth=200, cutoff=0.1, kernel="gaussia

filename_lr = root_data / "visium_hne_liana_bivariate.h5ad"

if filename_lr.exists():
    lrdata = ad.read_h5ad(filename_lr)
else:
    lrdata = li.mt.bivariate(
        adata, resource_name="mouseconsensus",
        local_name="cosine",      # local (per-spot) LR co-localisation sc
        global_name="morans",    # global spatial autocorrelation per LR
        n_perms=100, mask_negatives=False, add_categories=True,
        nz_prop=0.2, use_raw=False, verbose=True,
```

```
)
# lrdata: spots x LR-pairs AnnData; .var has morans / morans_pvals; .layer
top_global = lrdata.var.sort_values("morans", ascending=False)
```

```
In [29]: print(top_global.shape)
top_global.head(6).T
```

```
(454, 10)
```

```
Out[29]:
```

interaction	Nptx1^Nptxr	Rims1^Slc17a7	Cck^Cckbr	Sema4d^Plxnb3	Rtn4^Lingo
ligand	Nptx1	Rims1	Cck	Sema4d	Rtn4
receptor	Nptxr	Slc17a7	Cckbr	Plxnb3	Lingo
ligand_means	1.045611	0.628196	2.622576	0.544131	2.44654
ligand_props	0.723586	0.638393	0.947917	0.524554	0.99218
receptor_means	1.783589	2.257496	0.295722	0.190867	1.55361
receptor_props	0.889509	0.891741	0.344494	0.219866	0.89360
morans	0.434961	0.292247	0.243198	0.234733	0.21780
morans_pvals	0.0	0.0	0.0	0.0	0.0
mean	0.768262	0.708129	0.489883	0.385211	0.90848
std	0.197387	0.170169	0.263988	0.243879	0.08102

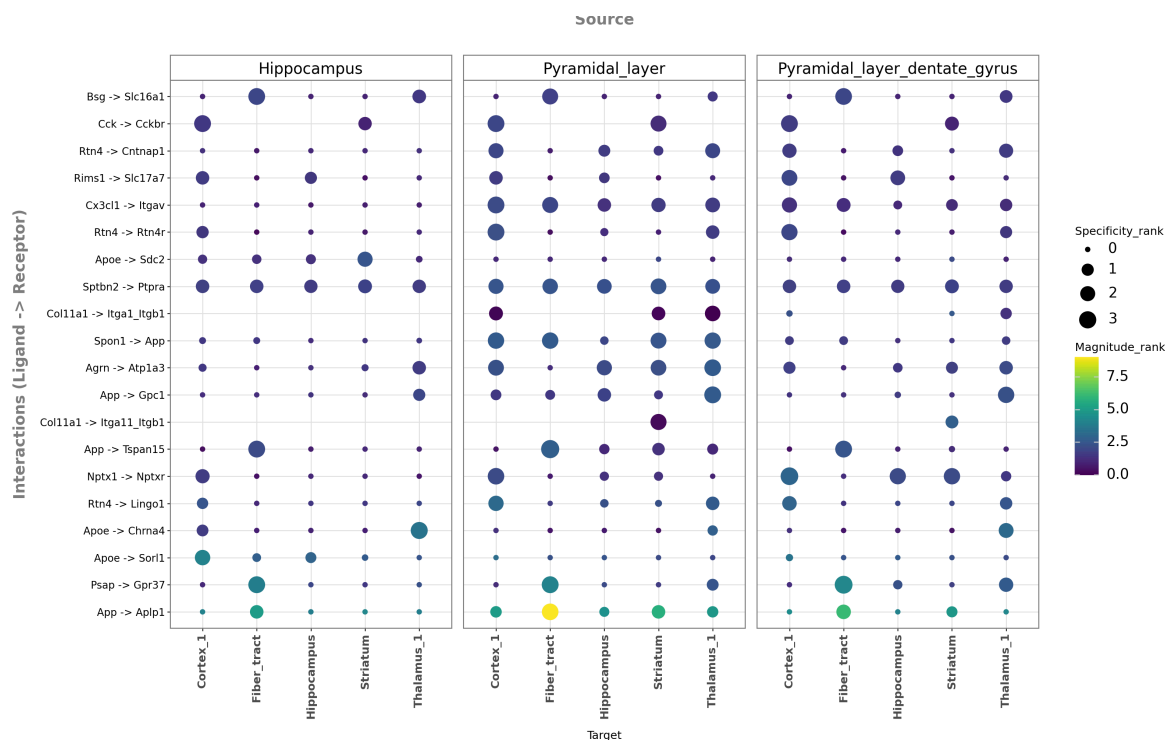
spatial (bivariate) results: the ligand^receptor name is the INDEX, so keep it

```
In [40]: fname = "liana_spatial_global_morans.tsv"
filename = root_result / fname

df_mor = lrdata.var.sort_values("morans", ascending=False)

if not filename.exists():
    pdwritecsv(df_mor, fname, root_result, index=True, verbose=True)
    print(f"File written: {filename}")

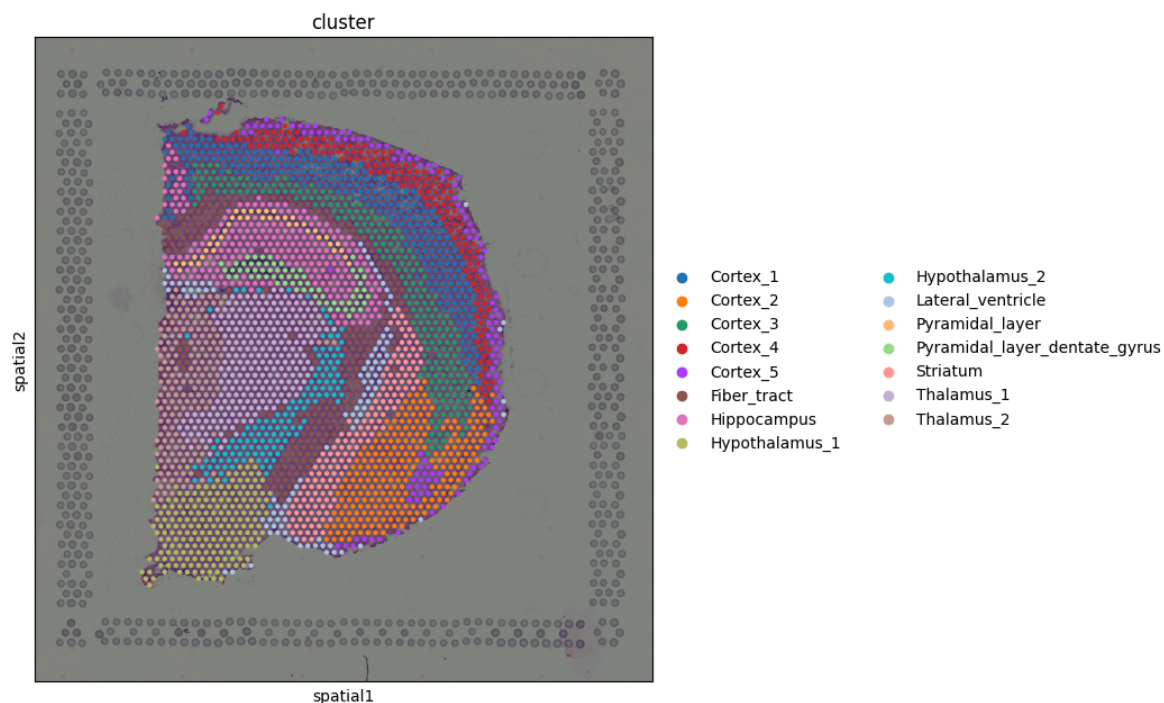
df_mor.head(6).T
```

Squidpy and Scanpy plots.

Get the axes back with `return_ax=True`, then save the parent figure:

```
In [45]: ax = sq.pl.spatial_scatter(adata, color="cluster", return_ax=True)
plt.show()
```



```
In [49]: lista = df_mor.head(8).index.to_list()
lista
```



```
Out[49]: ['Nptx1^Nptxr',
          'Rims1^Slc17a7',
          'Cck^Cckbr',
          'Sema4d^Plxnb3',
          'Rtn4^Lingo1',
          'Rtn4^Rtn4r',
          'Lgi1^Rtn4r',
          'Fgf1^Fgfr2']
```

```
In [ ]: # several panels -> a list of axes; all share one figure
        ligand_receptor_list = lista

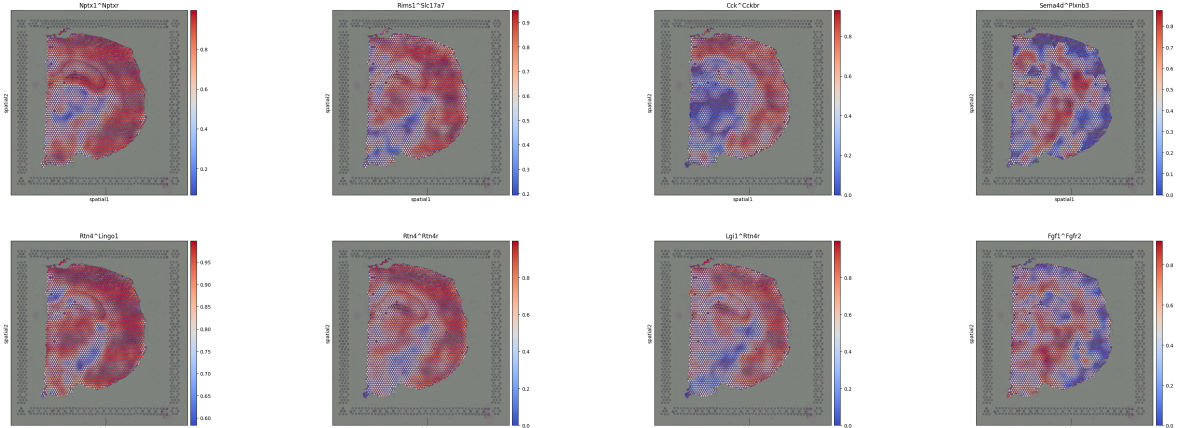
        axs = sq.pl.spatial_scatter(lrdata, color=ligand_receptor_list, cmap="coo

        filename = root_figure / "liana_spatial_LR.png"

        if not filename.exists():
            axs[0].figure.savefig(filename, dpi=300, bbox_inches="tight")
            print(f"Plot saved: {filename}")

        plt.show()
```

Plot saved: /home/flavio/uv/squidpy/figures/liana_spatial_LR.png



Plot: two things to watch for:

1. Save before `plt.show()`. In Jupyter, `show()` closes the figure, so saving afterwards writes a blank image.
2. `bbox_inches="tight"` matters. Without it, legends placed outside the axes, like the ones you move below the plot in cell 25, get cut off.

```
In [ ]:
```

Save spatial LIANA object

- That way you don't have to rerun `li.mt.bivariate`.
- `adata.uns["liana_res"]` is already saved inside `visium_hne_complete.h5ad` if you write that file after running LIANA.

```
In [ ]: force=False

        if not filename_lr.exists() or force:
```

```
lrdata.write_h5ad(filename_lr)  
print(f"Data written to {filename_lr}")
```

Data written to /home/flavio/uv/squidpy/data/visium_hne_liana_bivariate.h5ad

In []: